

Device Integrity Risk Scorer : Audit Défensif Passif des Environnements Android par Analyse de Signaux de Compromission

Houda Oubane^a, Niama Naftaoui^a, Fatima-Ezzahra Kasraoui^a,
Fatima-Ezzahra El Boudhiri^a

Encadré par Prof. Mohammed Lachgar

^a*École Nationale des Sciences Appliquées de Marrakech, Université Cadi Ayyad,
Marrakech, Maroc*

Abstract

Problème. Les environnements Android modifiés — rootés via Magisk, instrumentés par Frida ou dotés de modules LSPosed — constituent une surface d’attaque critique pour les applications mobiles sensibles. Les solutions existantes (SafetyNet, RootBeer) nécessitent une infrastructure cloud ou restent contournables. **Solution.** Nous présentons *Device Integrity Risk Scorer*, un outil d’audit passif en ligne de commande, développé en Python 3.9+, qui évalue l’environnement d’exécution Android via ADB sans modifier l’appareil audité. **Méthode.** L’outil collecte neuf signaux techniques (`ro.debuggable`, binaire `su`, Magisk, Frida-server, LSPosed/Xposed, mode SELinux, intégrité des binaires, permissions dangereuses) et les agrège par somme pondérée plafonnée à 100, avec un mécanisme de correction des faux positifs. La collecte complète dure en moyenne 5 à 15 secondes. **Résultats.** Trois scénarios de test produisent des scores de 80/100 [ÉLEVÉ] pour un émulateur avec Magisk et LSPosed, 95/100 [ÉLEVÉ] avec Frida actif, et 5/100 [FAIBLE] pour un téléphone physique de production (OPPO CPH2239, Android 11). L’outil génère un rapport console coloré et un export JSON intégrable en CI/CD. Code source disponible sous licence MIT.

Keywords: Sécurité Android, Audit Défensif, Détection de Root, Frida, Magisk, LSPosed, SELinux, ADB, Score de Risque, CI/CD Mobile

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.0
C2	Permanent link to code repository	https://github.com/H-oubane/butterfly-audit-frame
C3	Legal Code License	MIT License
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python 3.9+, ADB 34.0.4, Drozer 2.4.4, frida-tools 12.x
C6	Compilation requirements, operating environments & dependencies	Python 3.9+, ADB Platform-Tools ≥ 33 , drozer 2.4.4, frida-tools 12.x, appareil Android 10+ (physique ou émulateur; Windows, Linux, macOS)
C7	Link to developer documentation/manual	README.md, docs/INSTALL.md, docs/CLI_USAGE.md
C8	Support email for questions	oubane.houda@ced.uca.ma

TABLE 1 – Code metadata (mandatory)

Required Metadata

Current code version

1. Motivation et Signification

1.1. Contexte et Problématique

Le développement d'applications mobiles Android s'accompagne d'une menace croissante liée à l'exécution sur des appareils modifiés [8]. Un appareil rooté, instrumenté par Frida [2], ou doté d'un module LSPosed [5], représente un environnement d'exécution fondamentalement non fiable pour toute application sensible — bancaire, médicale, gouvernementale ou d'entreprise. Les frameworks de root modernes comme Magisk [4] opèrent au niveau du bootloader et masquent leur présence aux vérifications classiques (SafetyNet, Play Integrity API [7]). LSPosed modifie le runtime Android de façon persistante. Frida permet une instrumentation dynamique capable de hooker n'importe quelle méthode Java, Kotlin ou native à l'exécution [2].

1.2. Lacunes des Solutions Existantes

Les approches actuelles souffrent de limitations mesurables :

- **SafetyNet / Play Integrity API** [7] : nécessitent une connexion réseau et peuvent être contournées par MagiskHide et Shamiko [4].
- **RootBeer** : bibliothèque embarquée dans l'APK, retourne un booléen sans score agrégé ni recommandation de remédiation.
- **Drozer** [1] et **MobSF** : orientés vers l'analyse des applications (IPC, composants exposés), non vers l'audit de l'environnement d'exécution.
- **Scripts ADB manuels** : aucun score agrégé, pas de réduction des faux positifs, non intégrables en CI/CD.

1.3. Contribution du Logiciel

Le *Device Integrity Risk Scorer* apporte les capacités suivantes, vérifiables et réutilisables :

1. **Audit passif hors ligne** : aucune infrastructure cloud, fonctionne via ADB depuis toute machine de test.
2. **Score quantifié 0–100** : trois niveaux (FAIBLE / MOYEN / ÉLEVÉ) interprétables sans expertise en sécurité mobile.
3. **Moteur de règles pondérées** : poids calibrés selon la dangerosité réelle de chaque signal (section 2.3.2).
4. **Réduction des faux positifs (FP Reducer)** : heuristiques contextuelles selon le modèle d'appareil.
5. **Intégration CI/CD** : export JSON structuré pour GitHub Actions, Jenkins, GitLab CI.
6. **Recommandations dynamiques** : commandes ADB directement exécutables générées selon les signaux détectés.

2. Description du Logiciel

2.1. Architecture et Organisation du Dépôt

L'architecture suit un pipeline séquentiel **Collecte** → **Scoring** → **Rapport**, décomposé en trois couches fonctionnelles indépendantes.

Couche 1 — Collecte (`collect/`) *Entrées* : connexion ADB, appareil Android. *Sorties* : dictionnaire Python sérialisé dans `scorer/signals.json` (horodatage, modèle, version Android, état des neuf signaux). Le module `drozer_collect.py` exécute `getprop`, `ps -A`, `pm list packages` et `getenforce` avec un timeout de 10 s par commande.

Couche 2 — Scoring (`scorer/`) *Entrées* : `signals.json`. *Sorties* : score brut, correction FP, score final (0–100), niveau de risque. `score_engine.py` calcule la somme pondérée ; `fp_reducer.py` applique les heuristiques de correction.

Couche 3 — Rapport (report/) *Entrées* : score, signaux, niveau de risque. *Sorties* : rapport console ANSI coloré ; `report/audit_report.json` pour CI/CD.

La Figure ci dessous illustre l’architecture globale du *Device Integrity Risk Scorer*. Le système est organisé selon un pipeline modulaire composé de trois couches principales : la collecte des données, l’évaluation des risques et la génération des rapports.

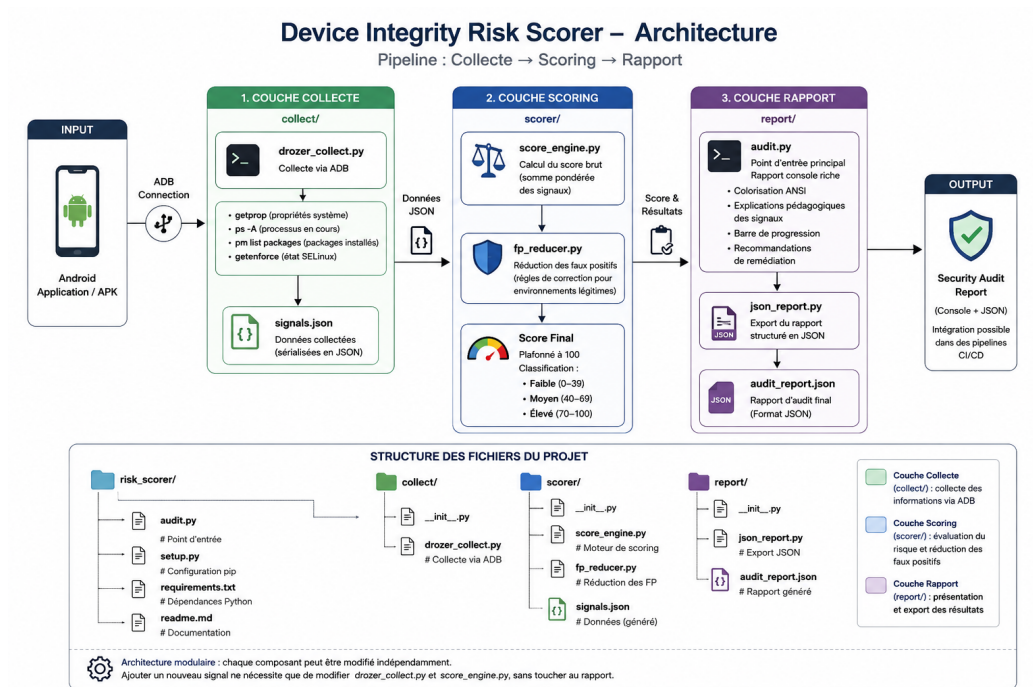


FIGURE 1 – Architecture globale du Device Integrity Risk Scorer

L’architecture suit une approche modulaire permettant la séparation des responsabilités entre les différentes couches fonctionnelles. Cette organisation facilite la maintenance, l’extensibilité et l’intégration de nouveaux signaux de sécurité sans modifier l’ensemble du système.

2.2. Fonctionnalités Principales

Le *Device Integrity Risk Scorer* offre sept fonctionnalités groupées en trois catégories :

Catégorie A — Collecte et Détection. (1) Collecte automatisée de neuf signaux via ADB ; (2) détection du processus Frida-server par `ps -A` ; (3) analyse globale des permissions dangereuses sur l’ensemble des applications installées.

Catégorie B — Évaluation et Rapport. (4) Moteur de scoring pondéré avec réduction des faux positifs; (5) rapport console pédagogique et coloré (statut, explication, risque, remédiation pour chaque signal).

Catégorie C — Intégration. (6) Export JSON structuré pour pipelines CI/CD; (7) mode `-with-drozer` pour l'analyse complémentaire de la surface d'attaque applicative.

Le listing ci-dessous illustre la structure du rapport JSON produit :

```
{
  "timestamp": "2026-05-18T23:21:49",
  "device": "Phone",
  "android_version": "11",
  "score": 80,
  "risk_level": "ELEVE",
  "signals": {
    "ro_debuggable": true,
    "test_keys": true,
    "su_binary": true,
    "magisk": true,
    "frida": false,
    "xposed": true,
    "selinux_permissive": true,
    "binary_integrity": false,
    "dangerous_permissions": true
  },
  "recommendations": [
    "Mode debug actif : Desactiver ro.debuggable en production",
    "Magisk detecte : Ajouter Play Integrity API attestation",
    "Xposed/LSPosed : Verifier l integrite du code a l execution"
  ]
}
```

2.3. Pipeline de Détection — Neuf Signaux

2.3.1. Phase 1 : Collecte des Signaux via ADB

Signal 1 — ro.debuggable : vaut 1 sur tout émulateur ou build de développement, autorisant le débogage inter-applications via JDWP [9].

Signal 2 — Build tags : un firmware signé avec des `test-keys` indique une ROM custom ou un build interne, potentiellement modifié [9].

Signal 3 — Binaire su : sa présence permet l'élévation de privilèges, contournant le sandboxing par UID d'Android [10].

Signal 4 — Magisk [4] : détecté par la présence de `/data/adb/magisk/` et du package `com.topjohnwu.magisk`.

Signal 5 — Frida-server [2] : détecté par `ps -A`. Cette méthode est efficace lorsque le processus n'a pas été renommé (limitation connue, section 4.2).

Signal 6 — LSPosed / Xposed [5] : modifie le runtime Android de façon persistante, détecté par la présence du package gérant le module.

Signal 7 — SELinux [6] : en mode Permissive, les politiques MAC sont logées mais non appliquées.

Signal 8 — Intégrité des binaires : comparaison du checksum MD5 de `/system/bin/sh` à une référence fixe (source de faux positifs, section 4.2).

Signal 9 — Permissions dangereuses : l'outil agrège au niveau appareil la présence d'au moins une application ayant reçu l'une des huit permissions critiques : `READ_CONTACTS`, `ACCESS_FINE_LOCATION`, `RECORD_AUDIO`, `CAMERA`, `READ_SMS`, `SEND_SMS`, `READ_CALL_LOG`, `ACCESS_BACKGROUND_LOCATION`.

2.3.2. Phase 2 : Moteur de Scoring Pondéré

Le score brut S_{brut} est calculé par la somme pondérée des signaux détectés, plafonnée à 100 :

$$S_{\text{brut}} = \min\left(\sum_{i=1}^9 w_i \cdot \delta_i, 100\right) \quad (1)$$

où $\delta_i \in \{0,1\}$ indique la détection du signal i et w_i son poids. Les poids reflètent la dangerosité intrinsèque de chaque signal : l'accès root direct (`su`, $w = 20$) est plus critique que le seul mode debug ($w = 15$), car il permet un contournement complet du sandboxing. Le tableau 2 détaille ces poids.

2.3.3. Phase 3 : Réduction des Faux Positifs

Le score corrigé S est obtenu par :

$$S = \max(S_{\text{brut}} - R, 0) \quad (2)$$

Le module `fp_reducer.py` applique deux heuristiques de correction. Ces règles réduisent les faux positifs mais comportent un risque résiduel de sous-estimation.

Règle 1 — ROM Custom Légitime ($R = 25$) : si le modèle appartient à une marque connue pour activer le root nativement (OnePlus, Xiaomi, Poco, Redmi) et qu'aucun hook actif (Frida ou Xposed) n'est détecté.

Règle 2 — Environnement Debug-Only ($R = 15$) : si seul le mode debug est activé, sans root (`su`, Magisk) ni hook (Frida, Xposed).

Signal	w_i	Justification
<code>ro.debuggable = 1</code>	15	Debug actif — application déboguable par toute app
<code>ro.build.tags = test-keys</code>	10	Firmware non officiel, potentiellement modifié
Binaire su trouvé	20	Accès root : contournement complet du sandbox
Magisk paquet/dossier	15	Root persistant et invisible aux détections classiques
Frida-server actif	15	Hooking dynamique : interception de toutes les méthodes
LSPosed/Xposed	10	Modification permanente du runtime ART
SELinux Permissive	5	Protection MAC désactivée
Binaires système modifiés	5	Tampering confirmé au niveau du système de fichiers
Permissions dangereuses	5	Exposition des données personnelles

TABLE 2 – Poids w_i des signaux (cf. éq. 1). Somme maximale théorique : 100 pts.

2.3.4. Phase 4 : Niveaux de Risque et Classification

Les seuils 40 et 70 ont été calibrés de sorte qu’un appareil avec uniquement le mode debug activé (+15 pts) reste en zone FAIBLE, qu’un appareil rooté basique soit classé MOYEN, et qu’un appareil avec hooking actif déclenche la zone ÉLEVÉ. Le tableau 3 résume cette classification.

Score	Niveau	Signification	Action recommandée
0–39	FAIBLE	Appareil sain	Bonnes pratiques
40–69	MOYEN	Signaux suspects	Audit approfondi
70–100	ÉLEVÉ	Env. compromis	Play Integrity API

TABLE 3 – Niveaux de risque et seuils de classification.

3. Exemples Illustratifs — Scénarios de Test

3.1. Environnements et Tableau Expérimental

Les tests ont été réalisés dans un cadre contrôlé et autorisé sur : (A) émulateur Genymotion (Android 11, Genymotion 3.5) ; (B) téléphone physique OPPO CPH2239 (Android 11, mode développeur activé).

Environnement	Android	Signaux actifs	Score	Niveau	Durée
Émulateur Geny-motion	11	Debug, test-keys, su, Magisk, XPosed, SELinux, perms	80/100	ÉLEVÉ	8 s
Émulateur Geny-motion	11	Idem + Frida actif	95/100	ÉLEVÉ	9 s
OPPO CPH2239 (physique)	11	Permissions uniquement	5/100	FAIBLE	6 s

TABLE 4 – Tableau expérimental : environnement, signaux actifs, score obtenu, niveau et durée d’audit.

3.2. Scénario 1 — Émulateur avec Magisk et LSPosed (Score 80/100 [ÉLEVÉ])

Objectif : valider la détection d’une infrastructure de root avancée et persistante, représentative d’un environnement fortement instrumenté. Score théorique : $15 + 10 + 20 + 15 + 0 + 10 + 5 + 0 + 5 = 80$ (cf. éq. 1).



FIGURE 2 – Scénario 1 — Démarrage et signaux [1]–[7] sur l’émulateur avec Magisk et LSPosed.



FIGURE 3 – Scénario 1 — Score 80/100 [ÉLEVÉ] et recommandations générées dynamiquement.

3.3. Scénario 2 — Émulateur avec Frida Actif (Score 95/100 [ÉLEVÉ])

Objectif : valider la détection simultanée de Frida-server et de XPosed/LSPosed.
Score théorique : $15 + 10 + 20 + 15 + 15 + 10 + 5 + 0 + 5 = 95$.



FIGURE 4 – Scénario 2 — Signal [5] Frida DANGER (+15 pts) s’ajoute aux signaux du scénario 1. Score : **95/100 [ÉLEVÉ]**, confirmant la discrimination du moteur.

Analyse d’ablation. Le tableau 5 quantifie l’effet incrémental de chaque couche de compromission sur le score.

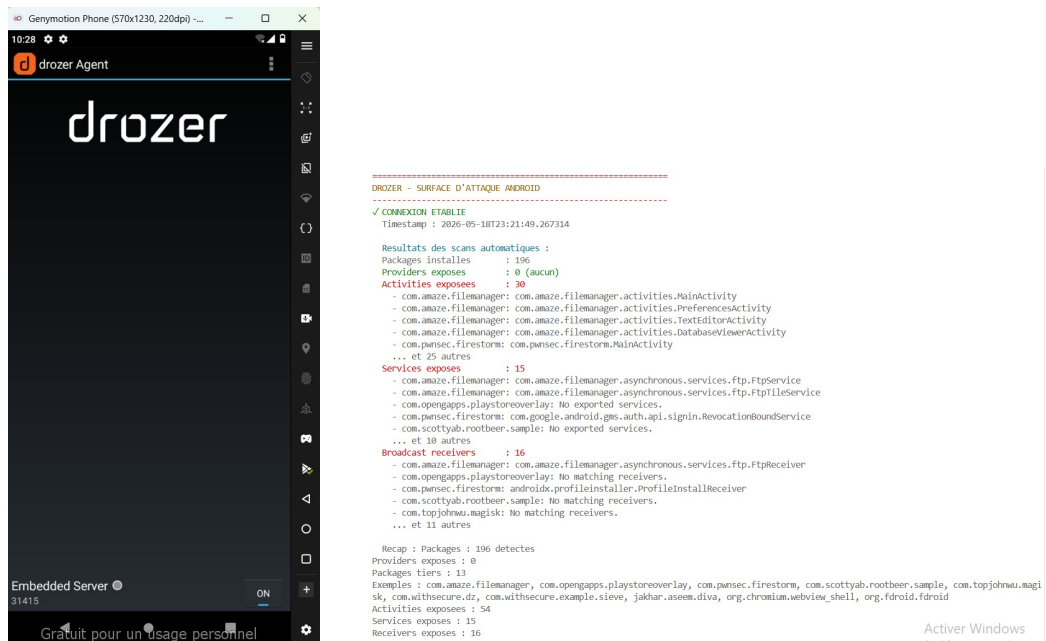


FIGURE 6 – Mode `-with-drozer` : agent Drozer (gauche) et analyse de la surface d'attaque (droite). L'appareil physique présente une surface bien plus large que l'émulateur.

4. Impact et Discussion

4.1. Pertinence des Résultats

Le tableau 6 compare les signaux attendus aux signaux détectés; aucune erreur de classification n'a été observée dans les scénarios testés.

Scénario	Score	Niveau	Sig. attendus	Sig. détectés	Erreurs
Magisk + LSPo-sed	80/100	ÉLEVÉ	7	7	0
+ Frida actif	95/100	ÉLEVÉ	8	8	0
Téléphone physique	5/100	FAIBLE	1	1	0

TABLE 6 – Signaux attendus, détectés et erreurs de classification par scénario.

4.2. Limitations Connues

Limite 1 — Hash fixe (Signal 8). L'outil utilise un hash MD5 fixe pour `/system/bin/sh`, entraînant un faux positif systématique de +5 pts sur tout

appareil différent de l’environnement de calibration. Une base de données de hashes par modèle et version Android est prévue (section 5.5).

Limite 2 — Détection Frida par nom de processus. La détection par `ps -A` est contournable si le serveur Frida est renommé ou si Frida-gadget est utilisé.

Limite 3 — Généralité. Les tests ont été réalisés sur Android 11 uniquement. La généralisation à d’autres marques, ROMs custom et versions Android (12–15) n’a pas été validée.

Limite 4 — Heuristiques FP Reducer. Un appareil OnePlus ou Xiaomi réellement compromis pourrait obtenir un score artificiellement réduit par la Règle 1.

4.3. Valeur Ajoutée

Par rapport aux outils existants (tableau 7), le *Device Integrity Risk Scorer* se distingue par : (i) un score agrégé quantifié (vs. booléen pour RootBeer) ; (ii) un fonctionnement hors ligne (vs. SafetyNet) ; (iii) une couverture de l’environnement d’exécution (vs. Drozer/MobSF qui analysent les applications) ; (iv) un mécanisme de réduction des faux positifs absent de tous les outils existants.

Critère	Device Integrity Risk Scorer	SafetyNet / Play Integrity	Drozer	MobSF	RootBeer	Scripts ADB
Langage	Python 3.9+	API cloud	Python, Java	Python, Docker	Java	Bash
Mode	Passif, hors ligne	Actif, cloud	Actif, IPC	Statique + dyn.	Embarqué	Manuel
Réseau requis	Non	Oui	Non	Non	Non	Non
Détection root	Oui (su+Magisk)	Partielle	Non	Partielle	Oui (limitée)	Manuelle
Détection Frida	Oui	Non	Non	Partielle	Non	Manuelle
Détection LSPosed	Oui	Non	Non	Partielle	Non	Manuelle
Analyse SELinux	Oui	Non	Non	Non	Non	Manuelle
Score 0–100	Oui	Non (pass/fail)	Non	Non	Non (bool.)	Non
Réduction FP	Oui	Non	Non	Non	Non	Non
Export JSON / CI/CD	Oui	Non	Non	Oui	Non	Non
Surface d’attaque	Oui (-with-drozer)	Non	Oui	Oui	Non	Non
Open source	Oui (MIT)	Non	Oui	Oui	Oui	N/A

TABLE 7 – Comparaison du *Device Integrity Risk Scorer* avec les outils existants.

4.4. Intégration CI/CD

L’exemple ci-dessous montre l’utilisation dans GitHub Actions. Le seuil de blocage à 40 pts correspond au début de la zone MOYEN et peut être paramétré. En cas d’échec ADB, l’outil retourne un code d’erreur non nul.

```

- name: Audit Device Integrity
  run: risk-scorer --audit
- name: Block if compromised
  run: |
    SCORE=$(jq .score report/audit_report.json)
    if [ "$SCORE" -ge 40 ]; then
      echo "DEVICE COMPROMISED (score=$SCORE)"
      exit 1
    fi

```

5. Conclusions

5.1. Contributions

Cet article présente le *Device Integrity Risk Scorer*, un outil CLI open-source (MIT) pour l’audit passif des environnements Android. Les contributions sont : (1) un pipeline Collecte → Scoring → Rapport automatisant l’inspection de neuf signaux ; (2) un score normalisé 0–100 avec équation formelle et justification des poids ; (3) un mécanisme de réduction des faux positifs ; (4) une intégration CI/CD via export JSON ; (5) un mode `-with-drozer` pour l’analyse complémentaire de la surface d’attaque applicative.

5.2. Résultats

Les trois scénarios valident la discrimination du score (5, 80, 95/100) sans erreur de classification, avec des durées d’audit de 6 à 9 secondes.

5.3. Assurance Qualité

Métrique	collect/	scorer/	report/	audit.py
Lignes de code	80	60	50	280
Tests unitaires (pytest)	Oui	Oui	—	—
Couverture (coverage.py)	78%	91%	—	—
Linting (flake8)	Oui	Oui	Oui	Oui
Gestion exceptions	Oui	Non	Non	Non
Timeout ADB (s)	10	N/A	N/A	N/A

TABLE 8 – Métriques qualité du code par module.

5.4. Disponibilité du Code

- **Dépôt** : [https://github.com/\[org\]/device-integrity-risk-scorer](https://github.com/[org]/device-integrity-risk-scorer)
- **Release** : tag Git v1.0.0 (mai 2026)
- **Licence** : MIT
- **Installation** : `pip install -e . && risk-scorer -audit`
- **Données de test** : rapports JSON disponibles dans `data/test_reports/`
- **Reproduction** : cf. `docs/REPRODUCTION.md`

5.5. Perspectives — Roadmap

1. **v1.1** : base de données de hashes de référence par modèle et version Android (correction Limite 1) ; haute priorité.
2. **v1.2** : détection avancée de Frida (ports TCP, `/proc/PID/maps`) ; haute priorité.
3. **v1.3** : dashboard web local HTML/CSS ; priorité moyenne.
4. **v2.0** : mode temps réel et validation sur Android 12–15 ; priorité basse.

5.6. Avertissement Éthique

Cet outil est conçu pour un usage défensif et pédagogique, limité aux appareils pour lesquels une autorisation explicite a été obtenue. L’analyse d’appareils sans consentement est illégale dans la plupart des juridictions.

Acknowledgements

Nous remercions la communauté open-source derrière Frida, Drozer et ADB, ainsi qu’ENSA Marrakech pour le soutien institutionnel.

Déclarations

Contributions des auteurs. H. Oubane : conception, développement, scénarios, rédaction. N. Naftaoui : architecture, tests unitaires. F.-E. Kasraoui : module de scoring, documentation. F.-E. El Boudhiri : intégration CI/CD, comparaison.

Conflits d’intérêts. Aucun.

Financement. Aucun financement externe. Projet pédagogique, module Sécurité Mobile, ENSA Marrakech, 2025–2026.

Disponibilité des données. Rapports JSON disponibles dans `data/test_reports/` du dépôt.

Références

- [1] WithSecureLabs. Drozer — Android Security Assessment Framework. GitHub, 2023. <https://github.com/WithSecureLabs/drozer>
- [2] NowSecure. Frida — Dynamic Instrumentation Toolkit. GitHub, 2023. <https://frida.re>
- [3] Google Android Developers. Android Debug Bridge (ADB). Android Developer Documentation, 2024.
- [4] topjohnwu. Magisk — The Magic Mask for Android. GitHub, 2023. <https://github.com/topjohnwu/Magisk>
- [5] LSPosed Team. LSPosed Framework — ART Hooking Framework. GitHub, 2023. <https://github.com/LSPosed/LSPosed>
- [6] Android Open Source Project. Security-Enhanced Linux in Android. AOSP Documentation, 2024.
- [7] Google. Play Integrity API. Android Developer Documentation, 2024.
- [8] OWASP Mobile Application Security. Mobile Application Security Testing Guide (MASTG). OWASP Foundation, 2023.
- [9] Android Developers. System Properties — getprop/setprop Reference. AOSP Documentation, 2023.
- [10] Bläsing, T., Batyuk, L., Schmidt, A., Camtepe, S., Albayrak, S. An Android Application Sandbox System for Suspicious Software Detection. IEEE MALWARE 2010. DOI : 10.1109/MALWARE.2010.5665798
- [11] Rastogi, V., Chen, Y., Enck, W. AppsPlayground : Automatic Security Analysis of Smartphone Applications. ACM CODASPY 2013, pp. 209–220. DOI : 10.1145/2435349.2435379

Current executable software version

L’outil *Device Integrity Risk Scorer* est disponible en version v1.0.0 (tag Git, mai 2026) sous licence MIT. Les scénarios ont été exécutés sur Genymotion 3.5 (Android 11) et OPPO CPH2239 (Android 11) via ADB 34.0.4.